

Assignment 2 – Deep Learning

Lior Kotlar – cse username: lior.kotlar, Ron Hafzadi - ronhafzadi

Task 1: Autoencoders

In this task, we implemented a convolutional autoencoder to reconstruct images from the MNIST dataset through a compressible latent representation. The goal was to design an architecture that is both flexible and capable of generalizing across various configurations.

- To ensure flexibility in latent space dimensionality, the encoder concludes with a linear fully-connected layer, which maps the final convolutional output (flattened) to any latent dimension d . Conversely, the decoder begins with an additional fully-connected layer, mapping the latent vector back into the shape expected by the transposed convolutional layers. This design decouples the convolutional layers from the latent size, allowing arbitrary values of d without structural changes to the convolutional layers.

- We explored two architectural settings:

Small configuration:

First layer: 4 channels, Intermediate layers: 5 and 6 channels.

Large configuration:

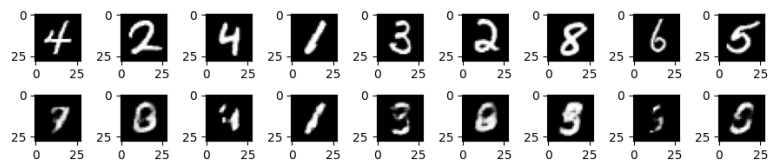
All convolutional layers use 16 channels.

Both configurations were tested with both $d = 4, d = 16$.

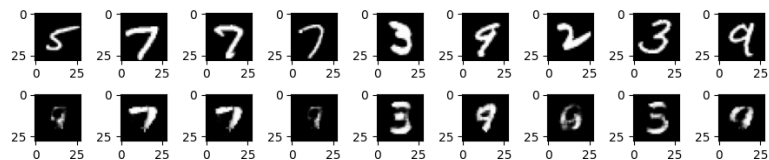
Here are the results of the reconstruction:

For the small model:

$d = 4$:

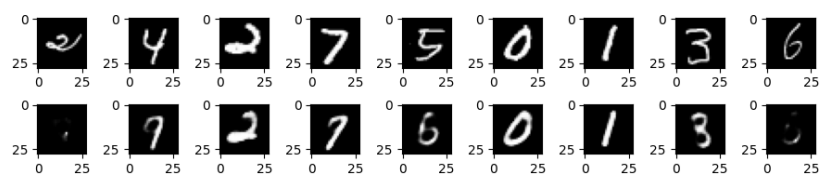


$d = 16$:

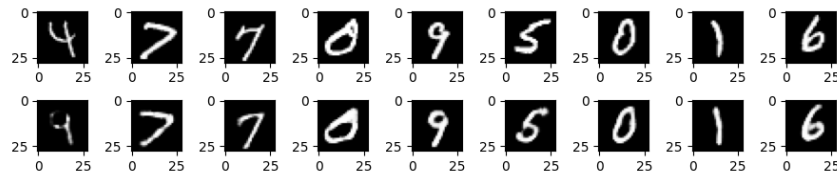


And for the large model:

$d = 4$:



$d = 16$:



When $d = 4$, the latent space is highly constrained, making it difficult for the model to encode sufficient information about the digit's structure. As a result, reconstructions are blurry and often lose fine-grained features such as curves and edges.

Increasing the latent dimension to $d = 16$ provides more capacity for encoding detailed and discriminative features. Although the quality is still not perfect, reconstructions preserve digit shape and structure much better, especially in the large network.

Network Architecture

- Encoder:
 - 3 convolutional layers with kernel size 3, stride 2, padding 1
 - Reduces 28×28 input to a 4×4 feature map (after 3 stride-2 convolutions)
 - `AdaptiveAvgPool2d((1, 1))` used to collapse spatial dimensions
 - Final Linear layer maps to latent dimension
- Decoder:
 - Starts with `Linear(latent_dim, third_layer_size)` to lift latent vector
 - Unflatten into shape $(C, 1, 1)$
 - `ConvTranspose2d` layers expand the representation:
 - First layer uses kernel size 7 to move from 1×1 to 7×7
 - Followed by two stride-2 transposed convolutions with padding and output padding to restore 28×28 resolution
 - Sigmoid activation at the output ensures pixel values are in $[0, 1]$
- Non-linearity: ReLU was used in all hidden layers for stable training and faster convergence.

This architecture provides a good balance of compression, feature extraction, and reconstruction capability, and is modular enough to extend for classification tasks in later parts.

Here is the code for the training loop:

```
def encoder_to_decoder_training(plot_save_directory,
                               latent_dimension,
                               model_size, pretrained_encoder_path = None,
                               save_encoder_path=None,
                               train_data_subset = False):
    (model, criterion, optimizer,
     scheduler, train_loader, test_loader) = get_training_tools(model_type=DECODER,
                                                                latent_dimension=latent_dimension,
                                                                model_size=model_size,
                                                                pretrained_encoder_path=pretrained_encoder_path,
```

```

train_data_subset=train_data_subset)

status_message = compose_status_message(DECODER, model_size,
pretrained_encoder_path, train_data_subset)
print(status_message)

outputs = []
train_losses, test_losses = [], []
learning_rate_updates_epochs = []
prev_lr = LEARNING_RATE
for epoch in range(NUM_EPOCHS):
    running_loss = 0.0
    for img, _ in train_loader:
        img = img.to(device)
        reconstruction = model(img)
        loss = criterion(reconstruction, img)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    epoch_loss = running_loss / len(train_loader)
    train_losses.append(epoch_loss)
    outputs.append((epoch, img, reconstruction))

model.eval()
test_running_loss = 0.0
with torch.no_grad():
    for img, labels in test_loader:
        img = img.to(device)
        reconstruction = model(img)
        loss = criterion(reconstruction, img)
        test_running_loss += loss.item()
test_loss = test_running_loss / len(test_loader)
test_losses.append(test_loss)

scheduler.step(test_loss)
current_lr = optimizer.param_groups[0]['lr']
if current_lr != prev_lr:
    print(f"Learning rate reduced from {prev_lr:.8f} to {current_lr:.8f}")
    learning_rate_updates_epochs.append(epoch)
    prev_lr = current_lr

print(f"Epoch {epoch + 1}/{NUM_EPOCHS} - Train Loss: {epoch_loss:.4f} | Test Loss:
{test_loss:.4f}")

plot_save_directory = plot_save_directory + ' d4' if latent_dimension ==
SMALL_LATENT_DIMENSION else (
    plot_save_directory + ' d16')

```

```

show_reconstructions_by_class(model, test_loader,
plot_save_directory=plot_save_directory)

intra_class_distance(test_loader, model.encoder, plot_save_directory=plot_save_directory)

plot_losses_and_reconstruction(outputs, train_losses, plot_save_directory, NUM_EPOCHS)

if save_encoder_path:
    torch.save(model.encoder.state_dict(), save_encoder_path)
    print(f'saved trained encoder model to {save_encoder_path}')

```

Task 2 : Classifier

In this task, we extended the encoder from Task 1 into a digit classifier for MNIST. Specifically, we reused the convolutional encoder architecture and added a simple fully connected layer (MLP) that maps the latent representation to 10 output classes. The model was trained using cross-entropy loss in two settings:

1. Using the entire training set.
2. Using a random subset of 100 training examples.

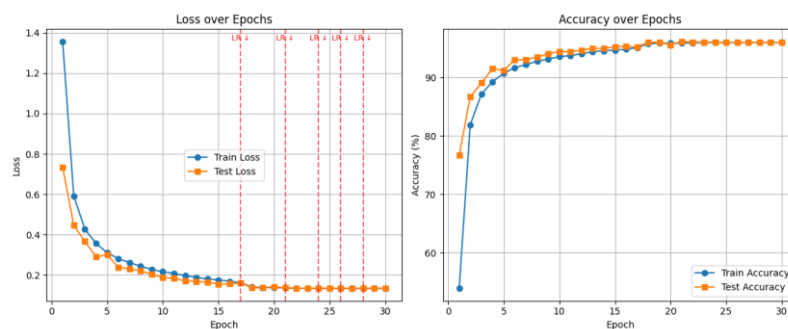
Classifier Design

We kept the architecture modular and clean by maintaining separate classes for:

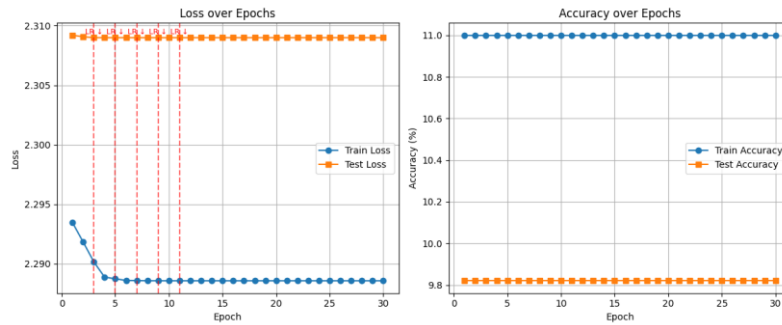
- Encoder: same encoder architecture from Task 1. Large model, $d = 16$.
- MLPClassifier: A fully connected layer that maps the latent vector to digit classes.
- EncoderDecoder: A wrapper that combines the encoder with either a decoder (for reconstruction) or an MLP (for classification), controlled using a `model_type` flag.

Here are the results in the end of the training:

Full training dataset:



Only a subset of the training data:



The model trained using the full training set reached a ~98% accuracy and the model trained on 100 examples achieved ~10% accuracy which is the expected accuracy from random guessing.

The red dashed line represents epochs in which there was a plateau and the learning rate was diminished.

Gap explanation:

With the full dataset (which is around 60,000 examples) the classifier had access to sufficient variety in handwriting styles, digit shapes, and noise patterns. This allowed the encoder to learn generalizable digit representations and the MLP to classify them accurately.

On the other hand, with only 10 samples per class on average (and possibly less due to randomness), the model could not learn meaningful features or general decision boundaries. This led to poor generalization and nearly random classification accuracy on the test set.

Here is the code of the training loop:

```
def encoder_to_classifier_training(plot_save_directory,
    latent_dimension = LARGE_LATENT_DIMENSION,
    pretrained_encoder_path = None,
    save_classifier_encoder_path = None,
    train_data_subset = False):
    if pretrained_encoder_path and save_classifier_encoder_path:
        print('ERROR: choose whether to use pretrained reconstruction encoder or to save
classifying encoder')
        exit(1)
    message = compose_status_message(CLASSIFIER, LARGE, pretrained_encoder_path,
train_data_subset)
    print(message)
    # if pretrained_encoder_path:
    #     print(f'starting to train encoder to classifier model, with a pretrained encoder
{pretrained_encoder_path}')
    # else:
    #     print('starting to train encoder to classifier model. training both encoder and classifier.')
    (model, criterion, optimizer,
scheduler, train_loader, test_loader) = get_training_tools(model_type=CLASSIFIER,
    model_size=LARGE,
    latent_dimension=latent_dimension,
    pretrained_encoder_path=pretrained_encoder_path,
```

```

train_data_subset=train_data_subset)
train_losses, test_losses = [], []
train_accuracies, test_accuracies = [], []
learning_rate_updates_epochs = []
prev_lr = LEARNING_RATE
for epoch in range(NUM_EPOCHS):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    for imgs, labels in train_loader:
        imgs, labels = imgs.to(device), labels.to(device)
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()*labels.size(0)
        _, predicted = torch.max(outputs, 1)
        correct += predicted.eq(labels).sum().item()
        total += labels.size(0)

    train_loss = running_loss/total
    train_accuracy = 100. * correct / total
    train_losses.append(train_loss)
    train_accuracies.append(train_accuracy)

    #=== testing
    model.eval()
    test_loss, correct, total = 0, 0, 0
    with torch.no_grad():
        for imgs, labels in test_loader:
            imgs, labels = imgs.to(device), labels.to(device)
            outputs = model(imgs)
            loss = criterion(outputs, labels)
            test_loss += loss.item()*labels.size(0)
            _, predicted = torch.max(outputs, 1)
            correct += predicted.eq(labels).sum().item()
            total += labels.size(0)
    test_loss = test_loss/total
    test_accuracy = 100. * correct / total
    test_losses.append(test_loss)
    test_accuracies.append(test_accuracy)

    scheduler.step(test_loss)
    current_lr = optimizer.param_groups[0]['lr']
    if current_lr != prev_lr:

```

```

print(f"Learning rate reduced from {prev_lr:.8f} to {current_lr:.8f}")
learning_rate_updates_epochs.append(epoch)
prev_lr = current_lr

print(f"Epoch {epoch + 1}/{NUM_EPOCHS} - Train Loss: {train_loss:.4f}, Acc:
{train_accuracy:.2f}% | "
      f"Test Loss: {test_loss:.4f}, Acc: {test_accuracy:.2f}%")

plot_loss_and_accuracy(train_losses, test_losses,
                       train_accuracies, test_accuracies,
                       plot_save_directory,
                       learning_rate_updates_epochs,
                       subset=train_data_subset)

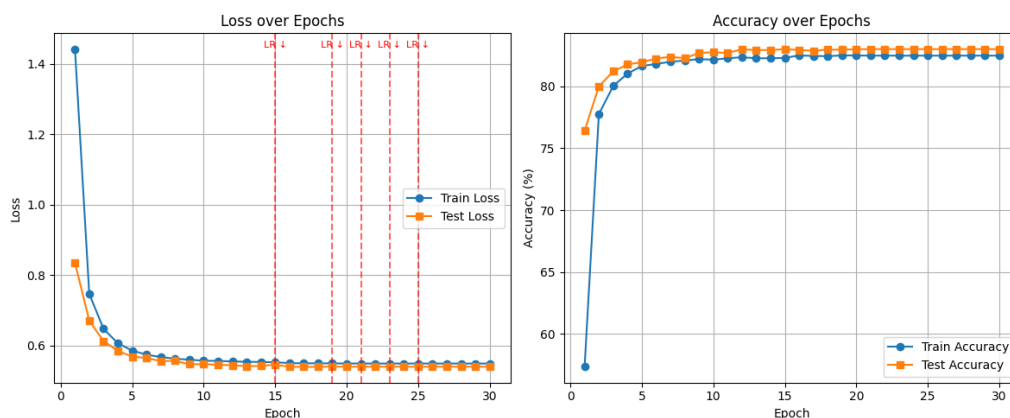
if save_classifier_encoder_path and not train_data_subset:
    torch.save(model.encoder.state_dict(), save_classifier_encoder_path)
    print(f'saved trained encoder model to {save_classifier_encoder_path}')

```

Task 3

While the encoder successfully captured some structural regularities in the MNIST data — as evidenced by the classifier reaching ~85% accuracy without any encoder fine-tuning — its performance fell short of the fully supervised model trained from scratch (~98% test accuracy).

Here are the graphs describing loss and accuracy across epochs. The model is the same as the model in task 2, with the large model architecture and $d = 16$, but using the fixed encoder weights from task 1.

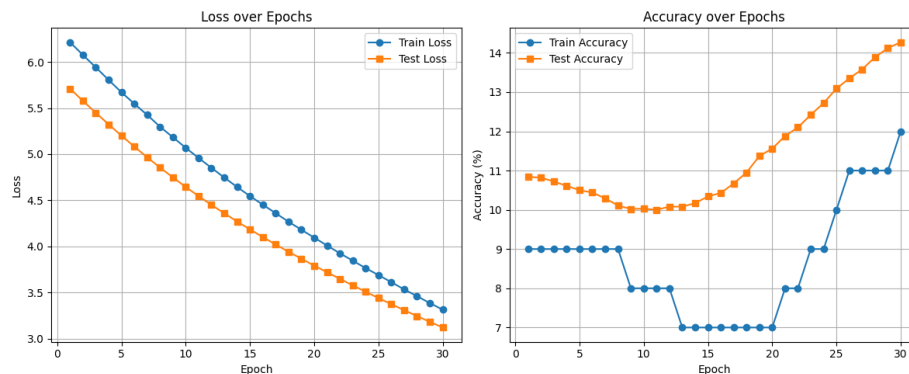


This performance gap suggests that:

Unsupervised representation learning is beneficial, as it allows reasonable classification performance without labelled supervision. However, task-specific supervision is crucial for optimal performance on classification tasks. Apparently, the classifier learned different subtleties and features than the reconstruction encoder. The representations learned during

reconstruction capture pixel-level similarity but may not be discriminative enough for digit classification without further fine-tuning.

And now, here are the results of the model using the encoder from question 1 and the classifier is trained over only 100 random examples from the training set:



With the entire dataset, the model achieves good classification accuracy and low loss. This indicates that the pretrained encoder has learned useful, general-purpose features that can be used effectively for classification.

However, with only 100 training examples, performance worsens significantly. The loss reaches a much higher value after the same number of epochs (30), and the accuracy behaves erratically: it briefly rises to ~10% (random chance for 10 classes), but then further to around 14%. This strange behaviour suggests overfitting and instability due to the tiny dataset, especially since the classifier is trying to map meaningful latent features without enough samples to learn from.

Task 4

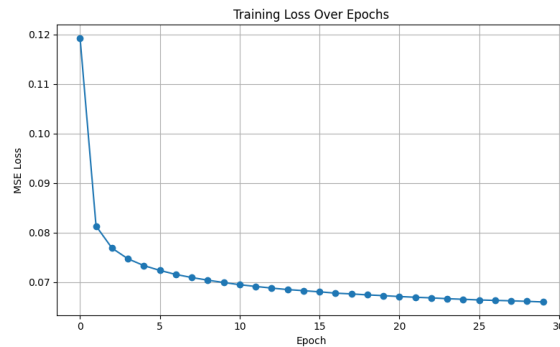
We compare two different encoder-decoder models for image reconstruction on MNIST:

Q1 Model: A conventional autoencoder trained end-to-end for reconstruction. Both the encoder and decoder are optimized together.

Q4 Model: A hybrid model where the encoder is pretrained for digit classification (from Task 2) and kept frozen, while only the decoder is trained for reconstruction.

Here is the performance review for the Q4 model:

First, training loss over epochs (the y axis is labelled 'MSE Loss but it's a mistake. It represents L1 Loss).



The Q1 autoencoder results in better reconstruction error — that is, it achieves a lower reconstruction loss compared to the Q4 hybrid model.

This is expected because in Q1, the encoder and decoder are trained together specifically to minimize reconstruction error. The encoder learns to produce latent representations that preserve spatial and pixel-level details relevant to image recovery, while the decoder learns to interpret those representations accordingly.

However, in Q4, the encoder is trained for classification, not reconstruction. It learns to ignore variations irrelevant to digit identity and to notice more abstract, discriminative features. These representations lack the detail necessary for high-quality reconstruction. Since the decoder is the only trainable part, it is forced to reconstruct from a latent space not designed for this task — resulting in higher reconstruction error overall.

In-class variability

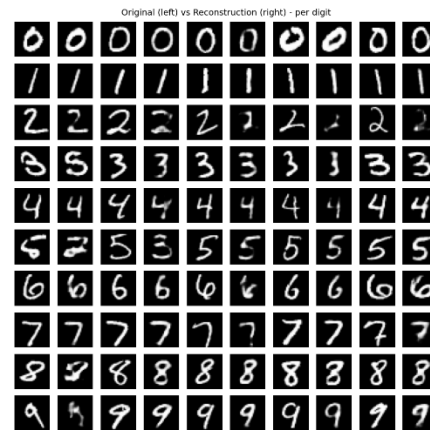
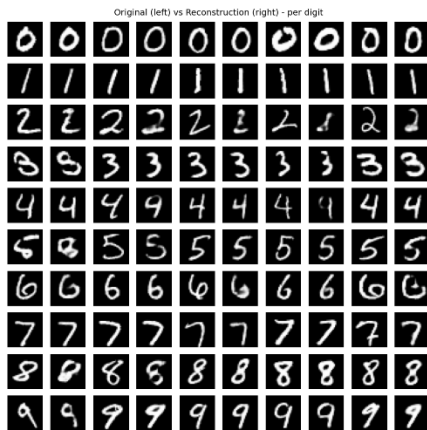
The Q4 model exhibits higher in-class variability. there is more visual variation among reconstructed images of the same digit class.

This happens because the frozen encoder used in Q4 was trained to make all instances of the same digit similar in latent space, regardless of their stylistic differences. For example, a thin '2' and a thick '2' may be encoded similarly, as long as they are classifiable as '2'. This compression of intra-class variation leads to loss of detail in the latent representation.

In contrast, the Q1 autoencoder considers subtle differences between inputs of the same digit, since it is trained to preserve all information necessary for pixel-level reconstruction, regardless of the digit label. Therefore, it tends to show more consistent reconstructions within each class.

Here are visualizations of the variability in each class. Each pair of images represents an original image and its reconstruction. Each lines represents a digit class.

The Q1 model is on the left, and the Q4 model is on the right.



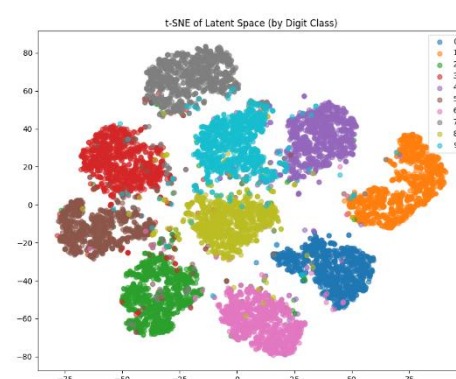
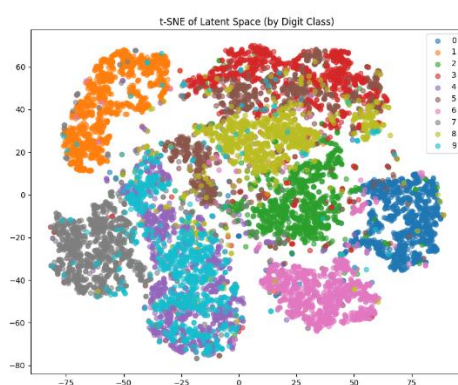
Intra-class distance

The Q4 model shows higher inter-class separation in the latent space.

This outcome is expected because the encoder in Q4 was trained specifically for classification. As a result, it learns to encode images of different digit classes to well-separated regions in latent space, maximizing discriminability between classes. This structured separation is clearly visible when visualized using t-SNE, where clusters corresponding to different digits are well-isolated.

In contrast, the encoder in Q1 is optimized only for reconstruction, not for class separation. Digits that share visual similarity (e.g., 3 and 8, or 4 and 9) may have overlapping latent representations. The latent space here is organized more around reconstructive similarity than semantic class boundaries, leading to lower inter-class separation.

Here are the latent vectors' t-SNE representation in model Q1 (on the left) and Q4 (on the right):



Deep Learning

Ex 2 Theoretical

May 13, 2025

Ron Hafzadi n Lior Kotlar
316482637 316219963

1. Q1

(1 Mark)

Show that a convolution with respect to any filter h is time/space invariant.

Ans:

Convolving with filter h is defined as:

$$y(t) = (x * h)(t) = \int_{-\infty}^{\infty} x(\tau)h(t - \tau) d\tau$$

shifting the input will yield $x'(t) = x(t - t_0)$, hence the output is:

$$y'(t) = (x' * h)(t) = \int_{-\infty}^{\infty} x'(\tau)h(t - \tau) d\tau = \int_{-\infty}^{\infty} x(\tau - t_0)h(t - \tau) d\tau$$

Now, substitute $\tau' = \tau - t_0$, gives:

$$y'(t) = \int_{-\infty}^{\infty} x(\tau')h((t - t_0) - \tau') d\tau' = (x * h)(t - t_0) = y(t - t_0)$$

2. Q2

(4 Marks)

Explain whether each of the following layers are time/space invariant or not:

- (a) Additive constant
- (b) Pointwise nonlinearity (such as ReLU)
- (c) Strided pooling by a factor
- (d) As a result, is a CNN composed of all these operators (+convolution) time invariant?

(4 Marks)

Ans:

- (a) Adding a constant c to the input is time/space invariant. If the input is shifted, the same constant is still added, resulting in the same shift in the output. i.e., for a system defined as:

$$y(t) = x(t) + c$$

where c is a constant, if we delay the input by d to get $x(t - d)$, the output becomes:

$$y_d(t) = x(t - d) + c = y(t - d)$$

- (b) Pointwise nonlinearity is time/space invariant. If the input is shifted, the function works on the same input, but shifted, making it the image of the function on the shifted input. i.e., For a pointwise nonlinearity f , where:

$$y(t) = f(x(t))$$

When the input is delayed by d , we get:

$$y_d(t) = f(x(t - d)) = y(t - d)$$

- (c) Strided pooling however, is not time/space invariant. suppose we take a maxpool in a window of 2 by 2, on the next image $[0, 0, 0, 0][0, 255, 255, 0][0, 255, 255, 0][0, 0, 0, 0]$. (if out of range, pad with zeros) then starting from index 0, will yield $[255, 255][255, 255]$, and starting from index 1 will yield, $[255, 0][255, 0]$, making the output dependent on time and space.
- (d) Taking into account what we have proven in the last parts, a CNN composed of convolutions, additive constants, pointwise nonlinearity, and strided pooling by a factor is not time/space invariant. removing the pooling layer however, will make it time/space invariant as composition of time/space invariant functions.

3. Q3

(3 marks)

Calculate the Jacobian matrix of the following layers:

- (a) Additive bias vector
- (b) General matrix multiplication
- (c) Convolution layer

(3 marks)

Ans:

(a) For a layer defined as:

$$y = x + b$$

where $x \in \mathbb{R}^n$ and $b \in \mathbb{R}^n$ are vectors, the output is given by:

$$y_i = x_i + b_i$$

Calculating the Jacobian:

$$J_{ij} = \frac{\partial y_i}{\partial x_j} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

which means

$$J = I_n$$

(b) For a fully connected layer:

$$y = Wx$$

where $W \in \mathbb{R}^{m \times n}$, the output is given by:

$$y_i = \sum_{j=1}^n W_{ij} x_j$$

Calculating the Jacobian:

$$J_{ij} = \frac{\partial y_i}{\partial x_j} = W_{ij}$$

which means

$$J = W$$

(c) Suppose the convolution is 1D with an input x of length n and a filter h of length k , The output is:

$$y_i = \sum_{j=0}^{k-1} h_j \cdot x_{i+j}$$

Calculating the Jacobian:

$$\frac{\partial y_i}{\partial x_j} = \begin{cases} h_{j-i} & \text{if } 0 \leq j - i < k \\ 0 & \text{otherwise} \end{cases}$$

which means

$$J = \begin{pmatrix} h_0 & h_1 & \cdots & h_{k-1} & 0 & \cdots & 0 \\ 0 & h_0 & h_1 & \cdots & h_{k-1} & 0 & \cdots \\ \vdots & & \ddots & & & \ddots & \\ 0 & \cdots & 0 & h_0 & h_1 & \cdots & h_{k-1} \end{pmatrix}$$